

“VIBRATION-BASED STRUCTURAL DAMAGE DETECTION USING DEEP LEARNING”

By
SRIJAN



Report Submitted to the
CSIR-SERC
CHENNAI, TAMIL NADU

Under the Guidance of
SHRI. SUBHAJIT DAS, SCIENTIST
STRUCTURAL HEALTH MONITORING LAB
CSIR-SERC, CHENNAI – 600113

DECLARATION

I, **SRIJAN**, hereby declare that this report is being submitted in fulfilment of the **PROF. G. S. RAMASWAMY SUMMER INTERNSHIP** at **CSIR-SERC, CHENNAI** and is result of the work carried out by me under the guidance of **SHRI. SUBHAJIT DAS, SCIENTIST, STRUCTURAL HEALTH MONITORING LAB, CSIR-SERC.**

I further declare that to my knowledge; the structure and content of this report are original and have not been submitted before any purpose.

SRIJAN

Reg. No.: 2019UGCE025

B. Tech. (Hons.), 3rd year (6th Sem.)

Civil Engineering

NIT JAMSHEDPUR

CERTIFICATE

This is to certify that **SRIJAN**, student of B. Tech. (Hons.) in Civil Engineering, from **National Institute of Technology, Jamshedpur** has successfully completed his summer internship on the project entitled “**VIBRATION-BASED STRUCTURAL DAMAGE DETECTION USING DEEP LEARNING**” under my supervision at **CSIR-SERC, CHENNAI**.

Date:

Place: Chennai

Signature of the Guide

SHRI. SUBHAJIT DAS

SCIENTIST

STRUCTURAL HEALTH MONITORING LAB

CSIR-SERC, CHENNAI

ACKNOWLEDGEMENT

Internship is an integral part of engineering curriculum providing students with first hand and practical aspects of their studies. It gives me great pleasure in completing my summer internship at CSIR-SERC and submitting the report for the same.

I would like to express the deepest appreciation to my guide **Shri. Subhajit Das**. He continually and convincingly conveyed a spirit of adventure regarding the thesis and research and an excitement regarding teaching. Without his guidance and endless support, it would not have been possible.

I would like to extend my sincere thanks to everyone who in some way or the other has provided me with valuable guidance, suggestion and help for this project.

ABSTRACT

Engineering structures are prone to various kinds of damages due to environmental or human-induced factors. Thus, early detection of structural damage is necessary for the structure to sustain and remain functional till its service life. Vibration-based damage detection methods utilize the vibration response of the monitored structure to assess its health; and identify and locate structural damage(s). Recently, various vibration-based damage detection techniques have been developed by using deep learning algorithms. In this report, we have used 1D CNN, LSTM and Bidirectional LSTM for damage classification on the experimental benchmark data of the Qatar University Grandstand Simulator (QUGS).

TABLE OF CONTENTS

Title	Page No.
1. Declaration.....	i
2. Certificate.....	ii
3. Acknowledgement.....	iii
4. Abstract.....	iv
5. Introduction.....	6
6. Convolution Neural Networks (CNNs).....	7
7. Long Short-Term Memory (LSTMs).....	9
8. Bidirectional LSTMs.....	12
9. Qatar University Grandstand Simulator (QUGS).....	13
10. Data Preparation.....	14
11. Hyperparameter Tuning.....	17
12. Representation of Models.....	18
13. Results and Conclusion.....	21
14. References.....	29

Introduction

The condition of civil infrastructure needs to be continuously monitored to check for presence of damage. Vibration-based or global structural damage detection techniques involve getting an overall idea about the health of the structure by converting the vibration response from the structure into signals. The signals measured by accelerometers are then processed and studied to know the presence, location, and severity of damage in the structure.

Vibration-based techniques are divided into parametric and non-parametric based approaches. In parametric methods, various algorithms are utilized to determine dynamic characteristics of the structure such as frequencies and modal shapes. Changes in these parameters with respect to the parameters identified for the undamaged case are used to recognize the structural damage. On the other hand, non-parametric approach involves statistical methods to directly identify damage.

A general process of damage detection involves getting the signals, then choosing important features and then classifying the structure based on those features. Machine learning methods combine the last two steps mentioned, thus reducing chances of human errors in the whole process.

Convolutional Neural Networks (CNNs)

They are a type of biologically inspired feedforward neural network that present a simple model for the mammalian visual cortex. They mainly consist of convolution blocks, pooling blocks, and fully connected blocks.

The convolutional block forms the main body of CNNs. It is where most computations take place. It consists of two matrices. The fixed input layer and set of trainable weights known as kernel. Convolution process involves the dot product of these two matrices. The kernels are of smaller dimension than input layer but larger in no. (depth). When moving forward, the kernel slides on the input matrix and performs dot product.

Convolution leads to sparse interaction, sharing of parameters and equivariance representation. Since kernels of smaller dimension than input layer, they can store important features in lesser no. of memory. Also, unlike traditional neural network, the weights in a kernel do not change if the points covered in the convolution change.

The pooling layer is like an abstract of an article. It is added after the convolution layer to sub-sample the convolution output. The pooling operation reduces the dimensionality of a given mapping while highlighting the prominent feature and it also helps to reduce overfitting. There are various pooling functions, most popular being maximum pooling where the maximum value amongst the pooling area is stored.

The fully connected layers are like the hidden layers of simple artificial neural networks and help in mapping the output of above operations in a single dimension. They are followed by output layers that produce the classification vector. CNNs are trained and optimized using backpropagation algorithm.

Since the inputs may not necessarily have linearity as the operations involved till now, we have activation functions which introduce non-linearity and help in better learning. To stop the weights from exploding, one can also use batch normalization layer in the network, which explicitly forces the weights in a kernel to take on unit Gaussian distribution.

CNNs are extensively used in object and event recognition systems in an image or video. Fig. 1 shows a general architecture of a 2D CNN model.

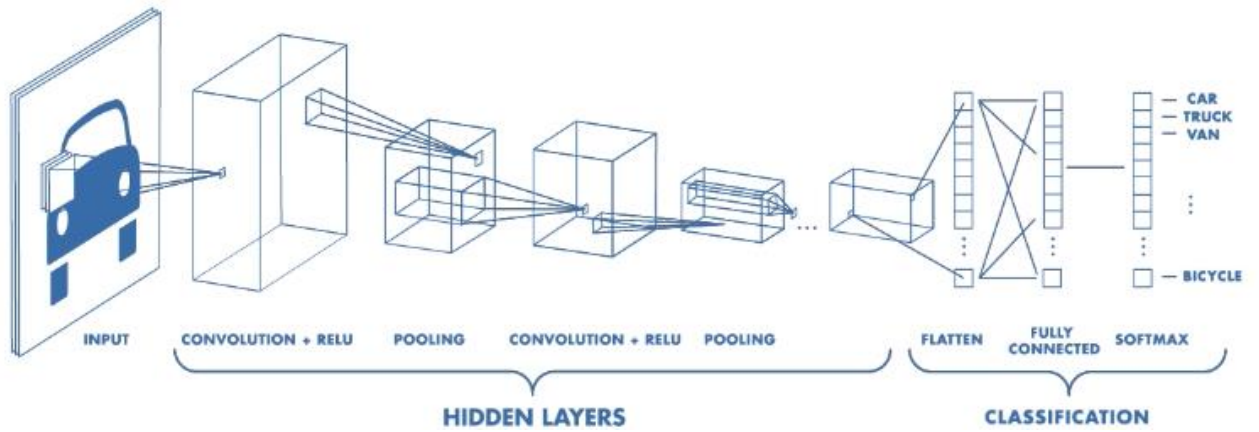


Fig. 1: Architecture of a 2D CNN. [4]

Here, we have used 1D CNNs for damage classification. There are differences between 2D CNNs and 1D CNNs. The main difference is the use of 1D arrays instead of 2D matrices for both kernels and feature maps. Accordingly, the 2D matrix manipulations such as 2D convolution (conv2D) have now been replaced by their 1D convolutions (conv1D). Moreover, the parameters for the kernel size and the sub-sampling are now scalars, and for the 1D CNNs, respectively. However, the fully connected layers are identical to their 2D counterpart and, therefore, it has the same backpropagation formulation.

Long Short-Term Memory (LSTMs)

Long Short-Term Memory networks – usually just called “LSTMs” – are a special kind of Recurrent Neural Networks (RNNs) that are capable of learning long-term dependencies. They can work on single data points (like images) as well as sequence of data (like signals, videos etc.) and hence find applications in variety of fields.

Traditional neural networks are not able to remember the connections between two data points which make them handicapped while handling sequences of data. RNNs handle them easily since they have networks with loops in them, allowing information to persist.

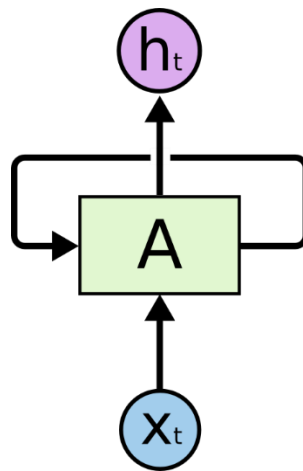


Fig. 2: Diagrammatic Representation of RNN. ^[5]

The neural network A, looks at some input x_t and outputs a value h_t . The loop allows information to be passed from one step of the network to the next.

The RNNs take input at a particular time t and use the output for that input, along with the input at time $t+1$ to generate next output and this process goes on for the complete sequence. Thus, they can use the outputs from past in their predictions for future. This can be easily understood by Fig. 2. The chain-like nature shows that RNNs are tightly bound with sequences and lists.

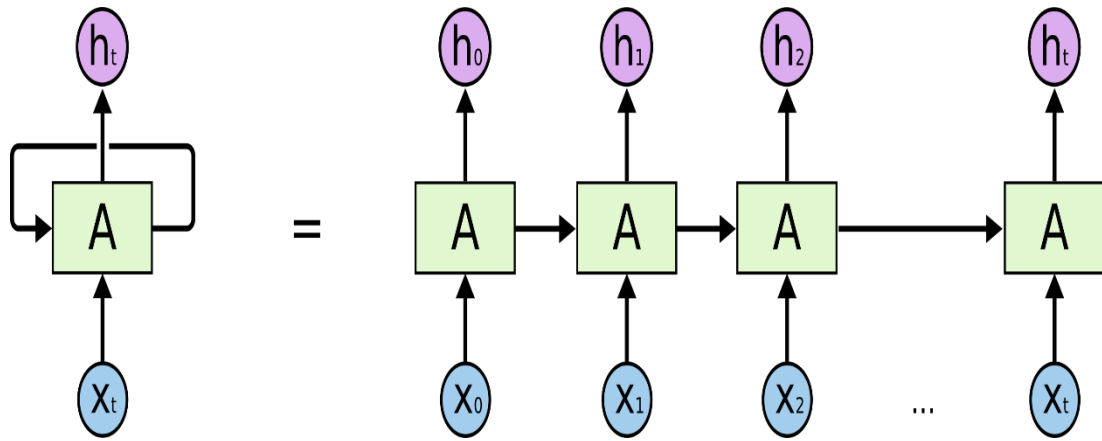


Fig. 3: Unrolled Representation of RNN. ^[5]

However, RNNs also bring their own set of issues such as exploding and vanishing gradients which make them difficult to solve problems that require learning long-term temporal dependencies i.e., selective reading, forgetting, and writing ability. LSTMs solve these issues (not exploding gradients) using interactions between *cell state* and a set of *gates*.

All recurrent neural networks have the form of a chain of repeating modules of neural network. In standard RNNs, this repeating module will have a very simple structure, such as a single *tanh* (a type of activation function) layer.

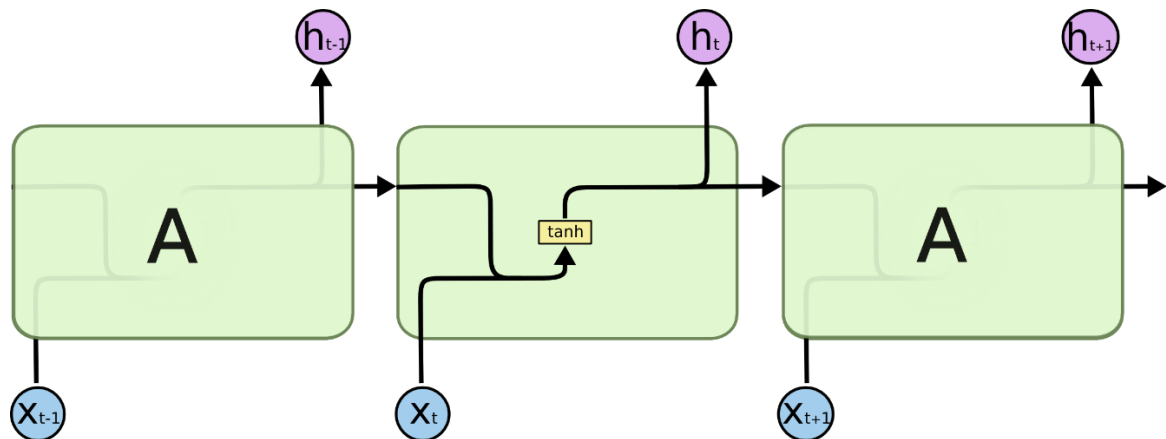


Figure 4: Repeating Module in RNN. ^[5]

LSTMs also have this chain-like structure but instead of having just a single layer, they have four different layers, each having its own function and interact among themselves in a unique way.

The key to LSTMs is the cell state, the horizontal line running through the top of the diagram in Fig. 5. It is like a conveyor belt. It runs straight down the entire chain, with only some minor linear interactions. It's very easy for information to just flow along it unchanged.

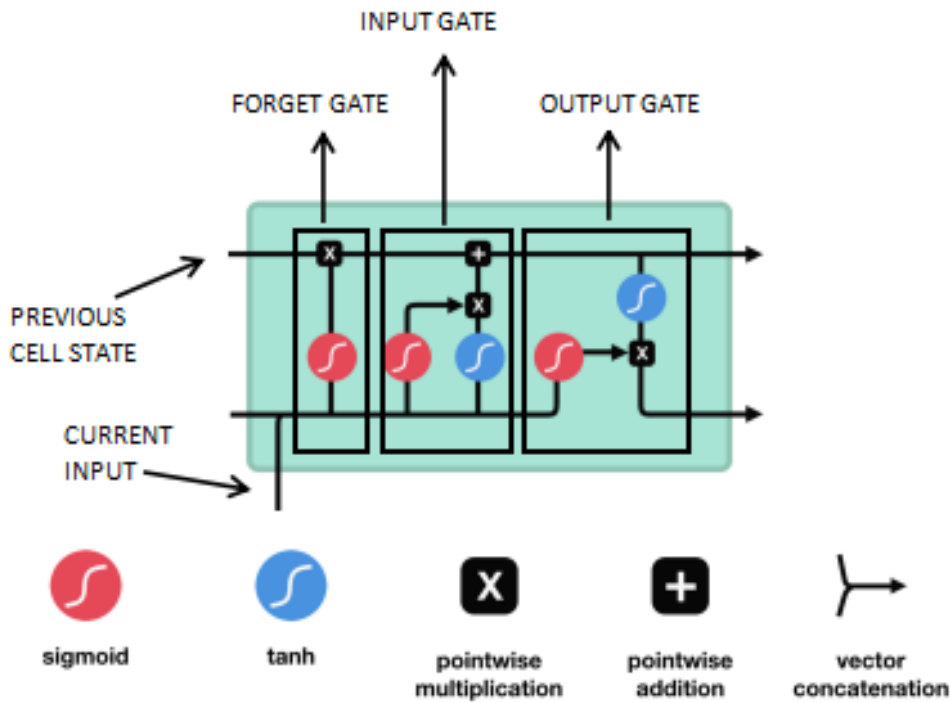


Figure 5: Repeating Module in LSTM.^[8]

The LSTM can carefully choose what information to add or remove to the cell state, carefully regulated by gates. There are three gates – input, forget and output.

The cell state encodes the information of the sequence observed up to that time step. The input gate controls the information added to the cell state from the current time step, and the forget gate controls what information needs to be forgotten from the current cell state. For example, if the output vector of the forget gate has a near-zero value in the first dimension, it indicates that the first dimension of the cell state needs to be “forgotten”. The forgetting occurs in the element-wise multiplication, i.e., multiplying an element by a near-zero value results in a near-zero element in the output vector. By maintaining cell state in this manner, the LSTM cell is able to capture both

long- and short-term relationships between the input time-series values and the predicted variable (e.g., damage classification). Finally, output gate decides what to output. This output will be based on the cell state but will be a filtered version.

Bidirectional LSTMs

Bidirectional recurrent neural networks are just putting two independent RNNs together. This structure allows the networks to have both backward and forward information about the sequence at every time step. One RNN moves forward, beginning from the start of the data sequence, and the other, moves backward, beginning from the end of the data sequence. They are used to make future predictions using both the information about the past and the future. The training of RNNs happen through an algorithm called backpropagation through time. The network blocks in a BRNN can either be simple RNNs or LSTMs.

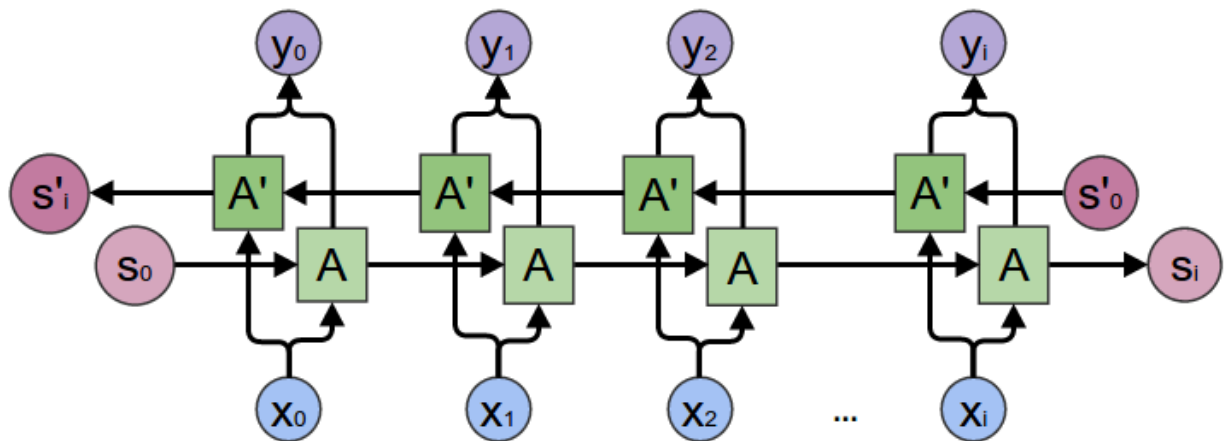


Fig. 6: Bidirectional RNNs.^[6]

Qatar University Grandstand Simulator (QUGS)

Damage detection where classification is done into damaged and undamaged class is called binary classification. In this report, QUGS experimental data^[3] has been used. QUGS has been developed for efficient structural damage detection techniques that are suitable for monitoring of modern stadia, as shown in Fig. 7. The frame was designed to carry a total of 30 spectators with area dimensions of 4.2 m $\times$ 4.2 m. The design considerations used for the experimental test structure was to guarantee its safety and compatibility with the specifications of modern grandstands. The structure was utilized in the form shown (steel frame only) to generate vibration data under several structural damage cases.



Fig. 7: QUGS testbed with joints numbered from 1 to 30.^[3]

A total of 31 scenarios are considered: in 30 scenarios, damage was introduced by loosening the bolts at the joints 1 to 30, respectively. And one scenario was undamaged case where all bolts of all 30 joints are tightened. A total of 30 accelerometers were mounted at the joint of each girder and the filler beam. The acceleration data was sampled at 1024 Hz and collected for 256 seconds.

Data Preparation

For each joint, data is divided into damaged and undamaged series. A single acceleration time series of a joint consists of many samples. The acceleration signals from both series are first normalized with respect to its mean and standard deviation. This improves the convergence rate of models trained on the datasets and prevents outliers from dominating the input. This is followed by windowing and sequencing (if required) and class balancing (under sampling).

N_d = no. of samples in damaged series

N_u = no. of samples in undamaged series

N_{dw} = no. of windows in damaged series

N_{uw} = no. of windows in undamaged series

N_{ds} = no. of sequences in damaged series

N_{us} = no. of sequences in undamaged series

f = sampling frequency (1024 Hz)

d = duration of signal recording (256 secs)

n = no. of joints (30)

w = no. of samples in a window

s = no. of windows in a sequence

Model	w	s
1D CNN	128	-
LSTM	64	2
Bi-LSTM	32	8

1D CNN

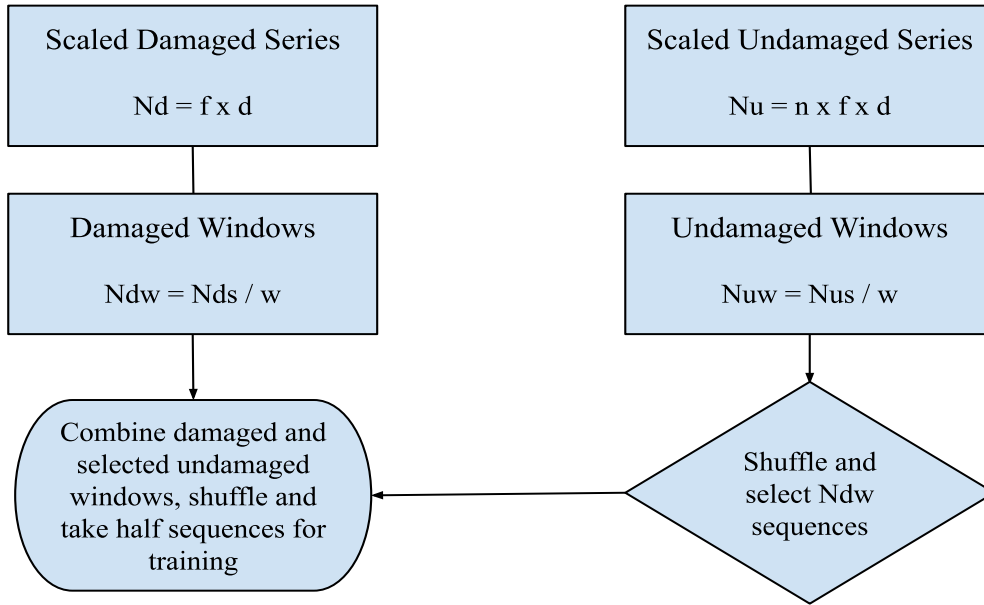


Figure 8: Data Preparation for CNN

LSTM and Bi-LSTM

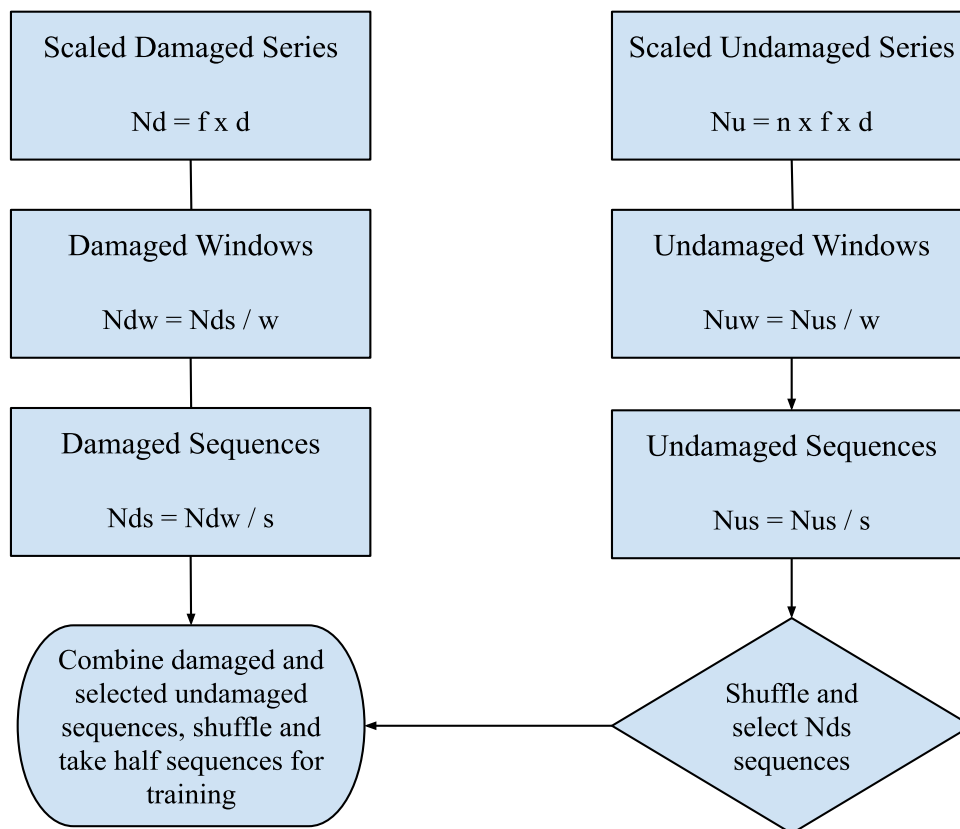


Figure 9: Data Preparation for LSTM and Bi-LSTM

Hyperparameter Tuning

A range of values for hyperparameters is used for tuning all three models using RandomSearch sub-class of Tuner class of Keras Tuner library. It uses random search algorithm which randomly selects a combination of values for hyperparameters from a specified set of possible values and trains the models with those parameters on splits of training and validation sets. Training is completed when, either a specified maximum number of epochs is reached, or the validation loss does not decrease for a specified number of epochs (i.e., early stopping). The final network weights are taken from the epoch with the smallest validation loss. The set of hyperparameters and their optimal combination is given in the following tables.

1DCNN

Hyperparameters	Values	Optimal Values
Window Size	32, 64, 128	128
Kernel Size	8, 16, 32, 64	64
No. of 1D CNN Layers	1 to 6	2
Filters	32, 64, 128, 256, 512	64, 512
Units in Dense Layer	8, 16, 32, 64	8
Learning Rate	$1e^{-2}$, $1e^{-3}$, $1e^{-4}$	$1e^{-4}$
Batch Size	16, 32, 64, 128	32

LSTM

Hyperparameters	Values	Optimal Values
Window Size	32, 64, 128	64
Sequence Size	2, 4, 8, 16	2
No. of LSTM Layers	1 to 6	5
Units in LSTM Layer	32, 64, 128, 256, 512	128, 64, 512, 64, 64
Units in Dense Layer	8, 16, 32, 64	64
Dropout Rate	0.1, 0.2, 0.3	0.1,0.2,0.3,0.2,0.3,0.2
Learning Rate	$1e^{-2}$, $1e^{-3}$, $1e^{-4}$	$1e^{-3}$
Batch Size	16, 32, 64, 128	16

Bi-LSTM

Hyperparameters	Values	Optimal Values
Window Size	32, 64, 128	32
Sequence Size	2, 4, 8, 16	8
No. of LSTM Layers	1 to 6	2
Units in LSTM Layer	32, 64, 128, 256, 512	64, 64
Units in Dense Layer	8, 16, 32, 64	64
Dropout Rate	0.1, 0.2, 0.3	0.3, 0.1, 0.3,
Learning Rate	$1e^{-2}$, $1e^{-3}$, $1e^{-4}$	$1e^{-4}$
Batch Size	16, 32, 64, 128	32

Representation of Models

1D CNN

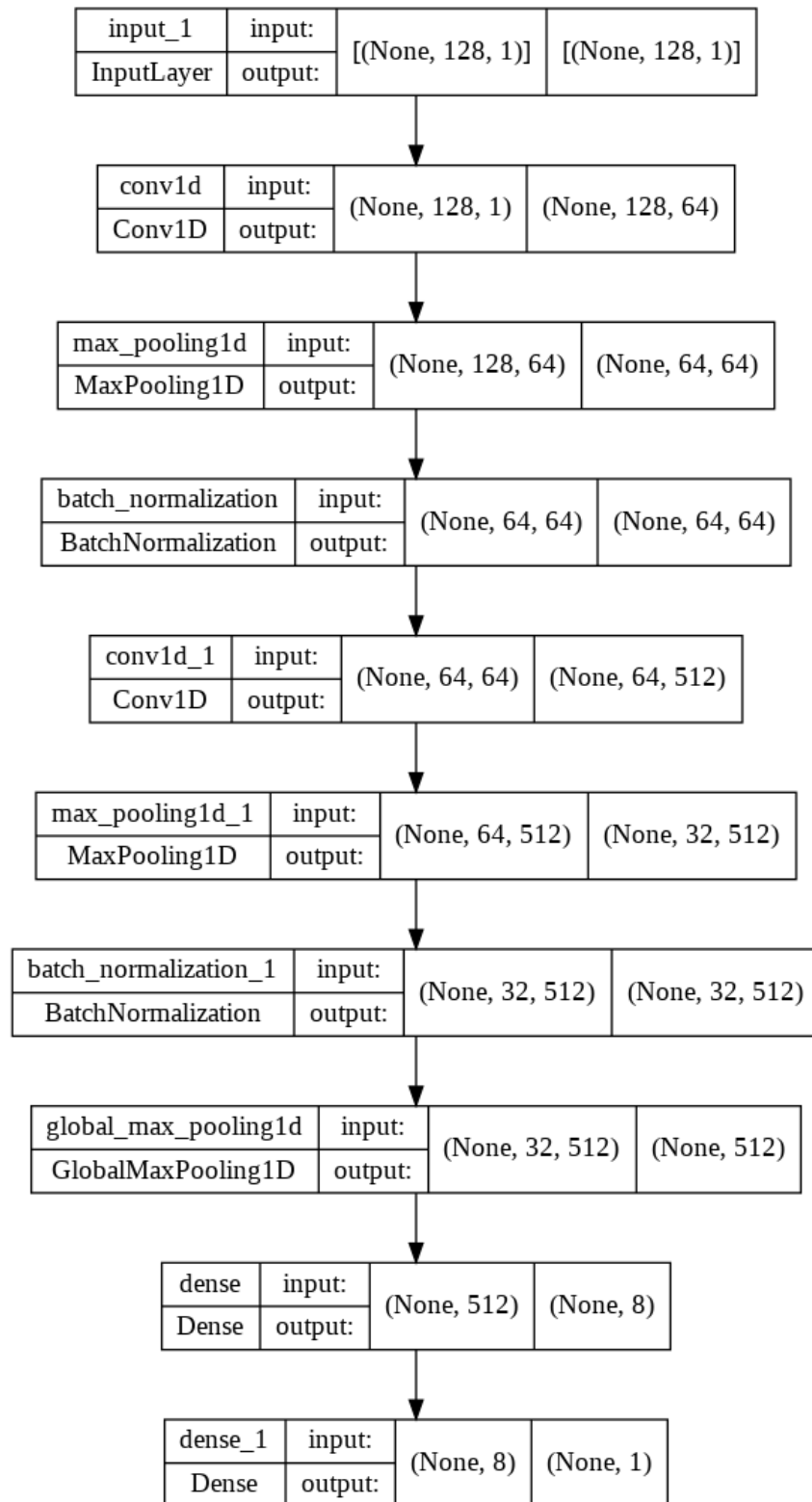


Fig. 10: Pictorial Representation of 1D CNN Model

LSTM

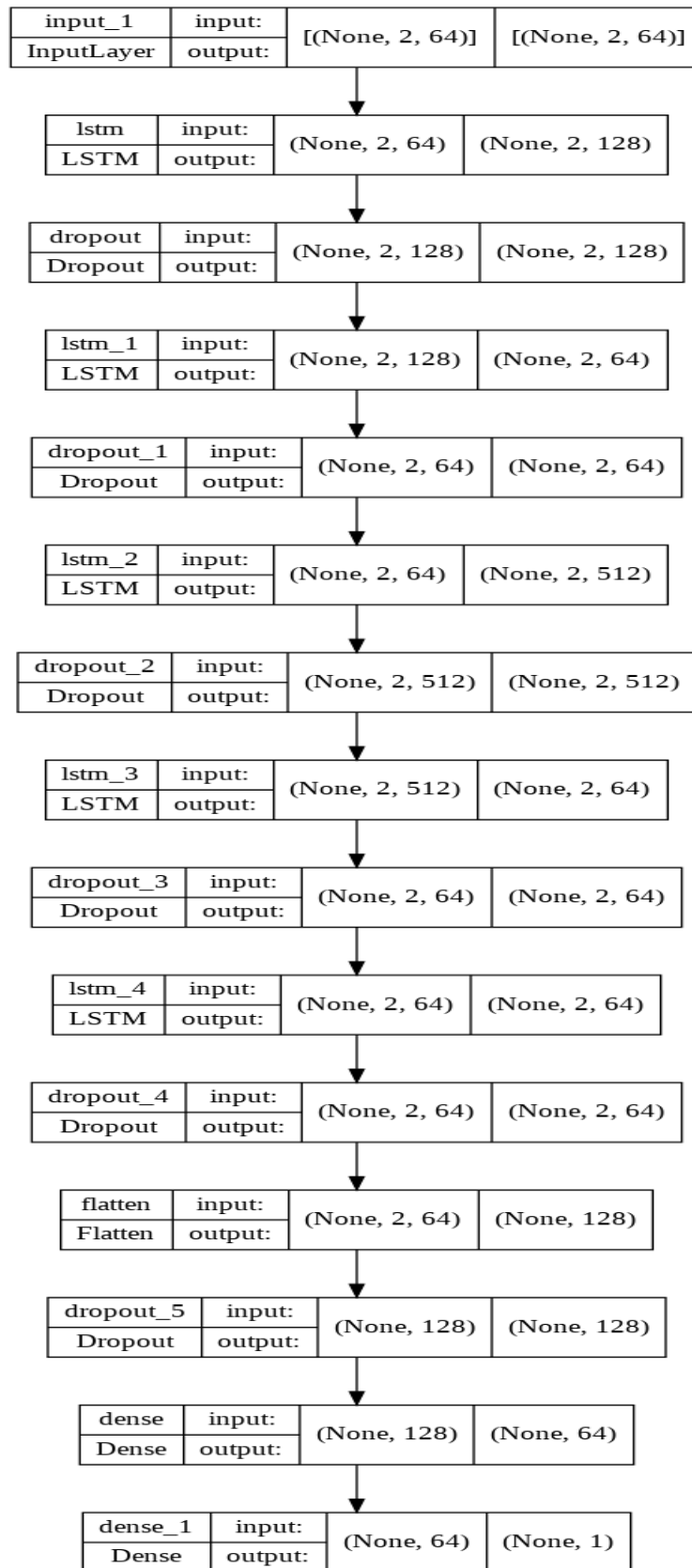


Fig. 11: Pictorial Representation of LSTM Model

Bi-LSTM

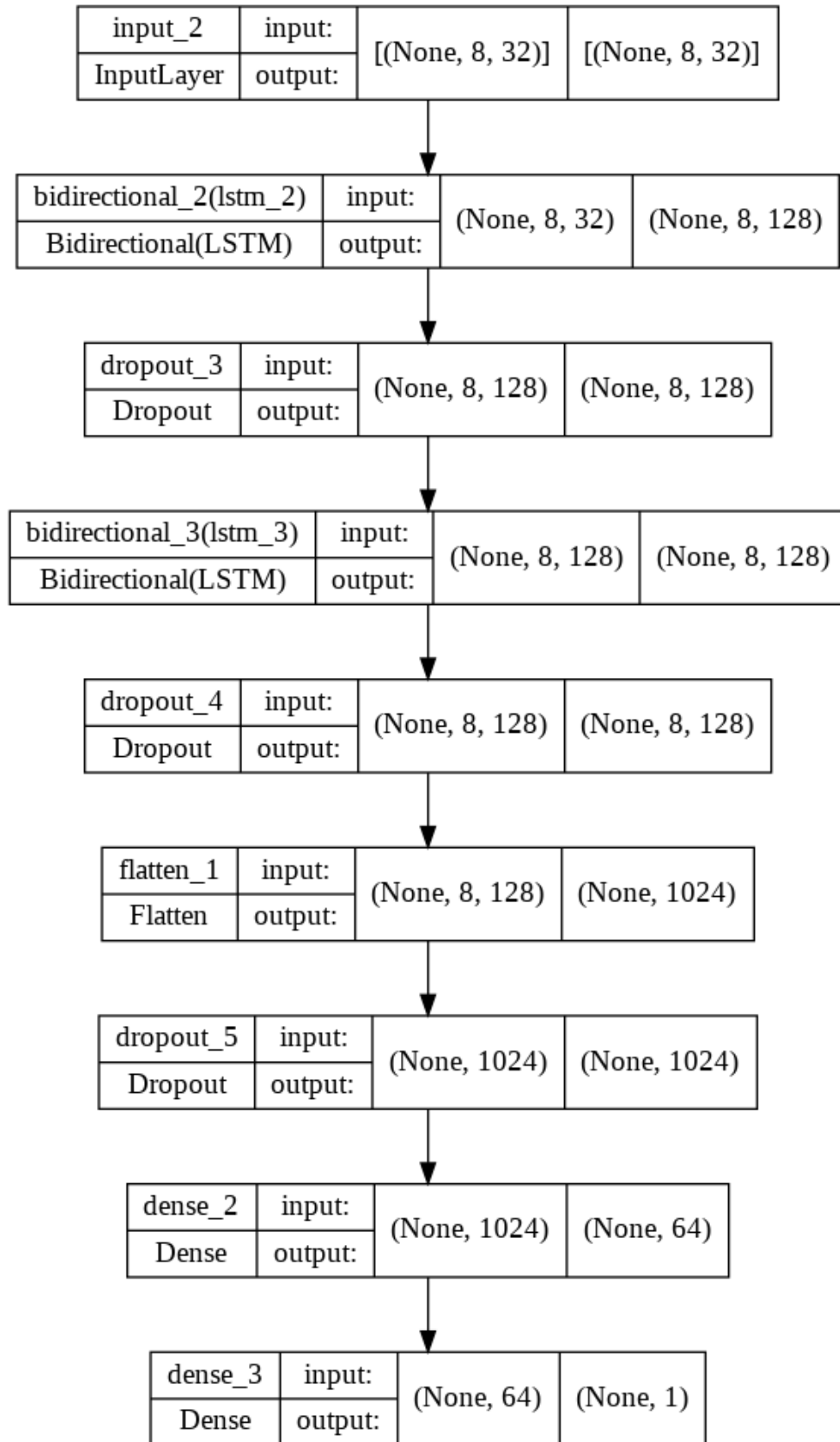


Figure 12: Pictorial Representation of Bi-LSTM Model

Results and Conclusion

Once the models are trained using optimal hyperparameter values, they are used to assess the condition of the structure. The model for a joint is used to compute an index that reflects the likelihood of damage at that joint, given by probability of damage (POD) and computed as below:

$$POD = \frac{D}{T}$$

where, D = no. of damaged windows (for 1D CNN) or sequences (for LSTM and Bi-LSTM), and

T = total no. of windows or sequences processed by the model.

POD computed at a damaged joint should be higher than that at an undamaged joint. This gives a clear indication regarding both the presence and the location of a structural damage. The distributions of PODs for multiple damage scenarios are presented as heatmaps for quickly localizing damage(s).

It can be inferred from the images that 1D CNN has given considerably better results than both LSTM and Bi-LSTM. Also, Bi-LSTM has performed better than LSTM.

However, it has to be further tested if the three models perform the same when the case is of multi-class classification, which are more prevalent in real-life scenarios.

Fig. Order - Top to Down: 1D CNN, LSTM, Bi-LSTM

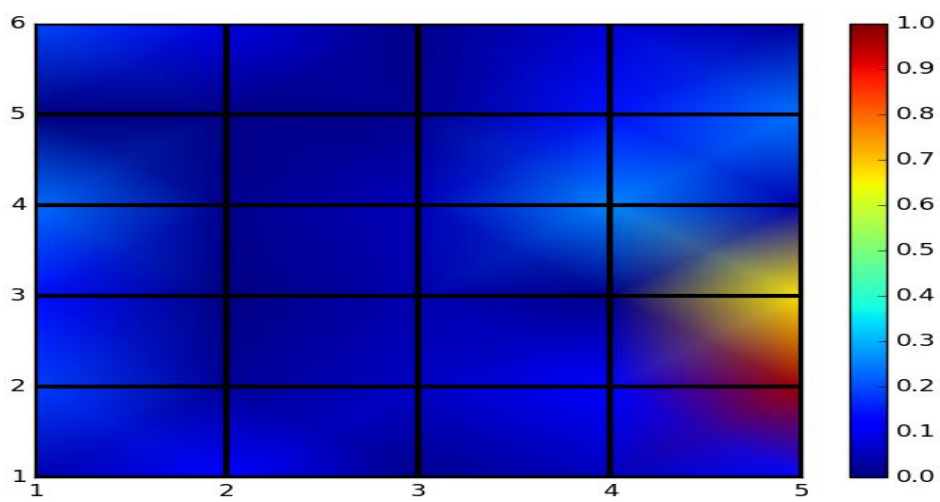
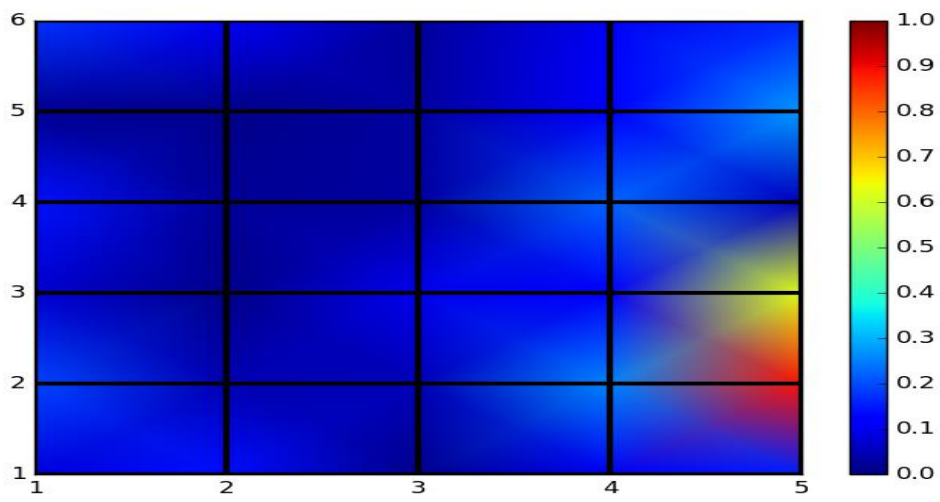
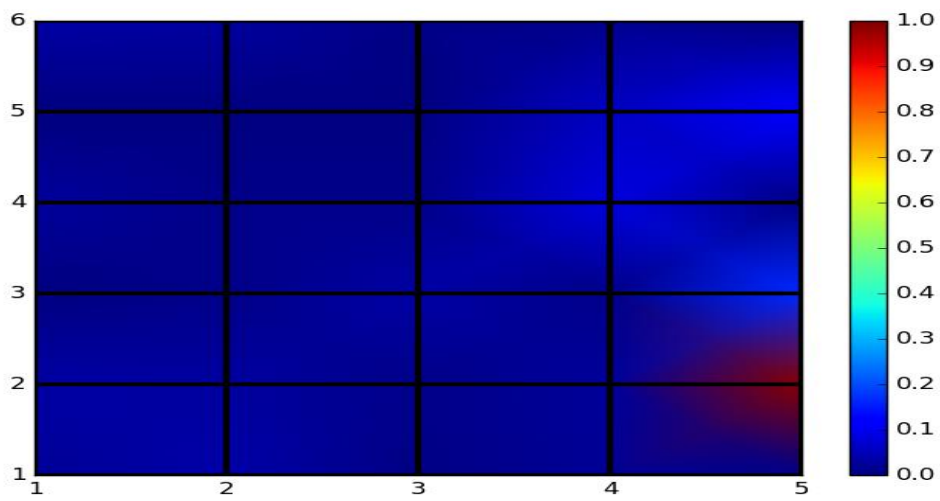


Fig. 13: Joint 10 Damaged

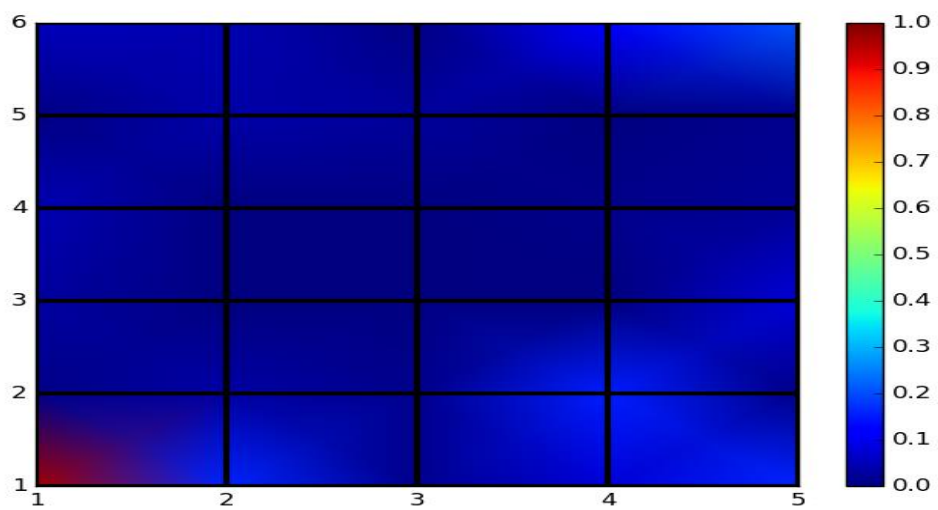
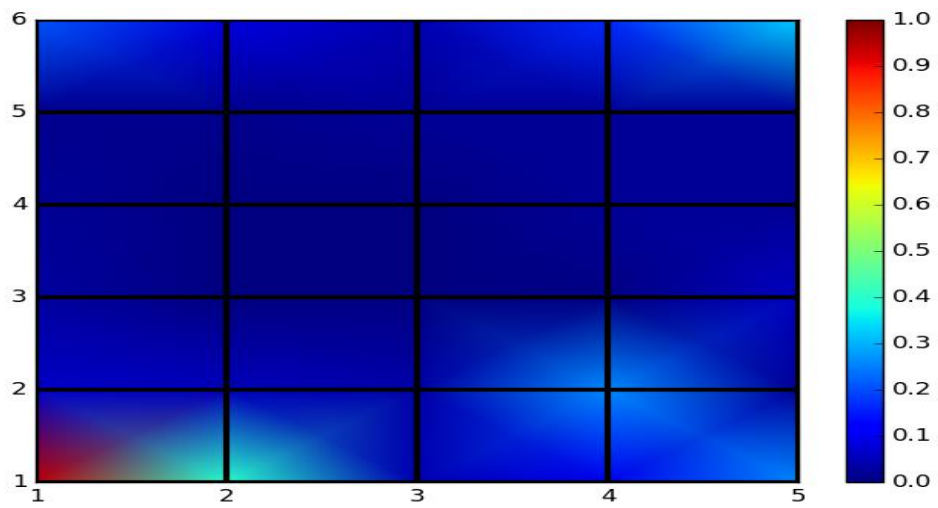
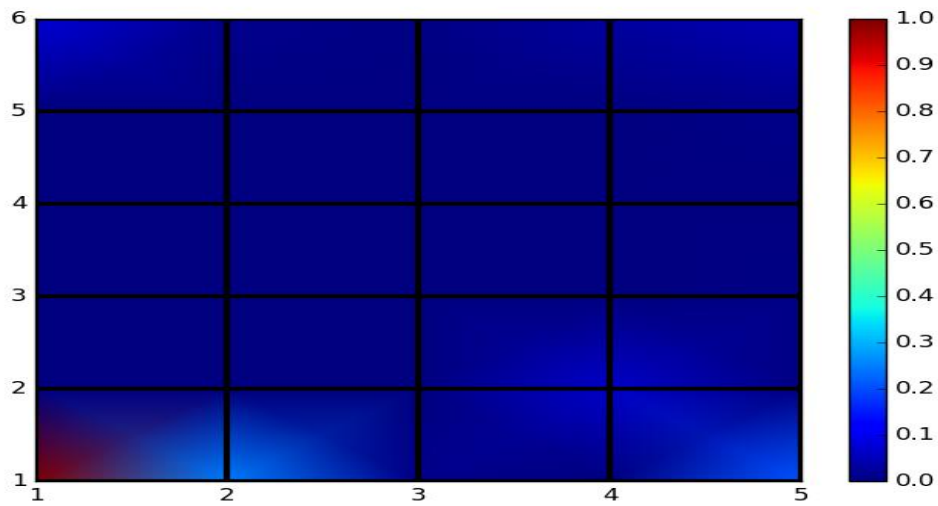


Fig. 14: Joint 1 Damaged

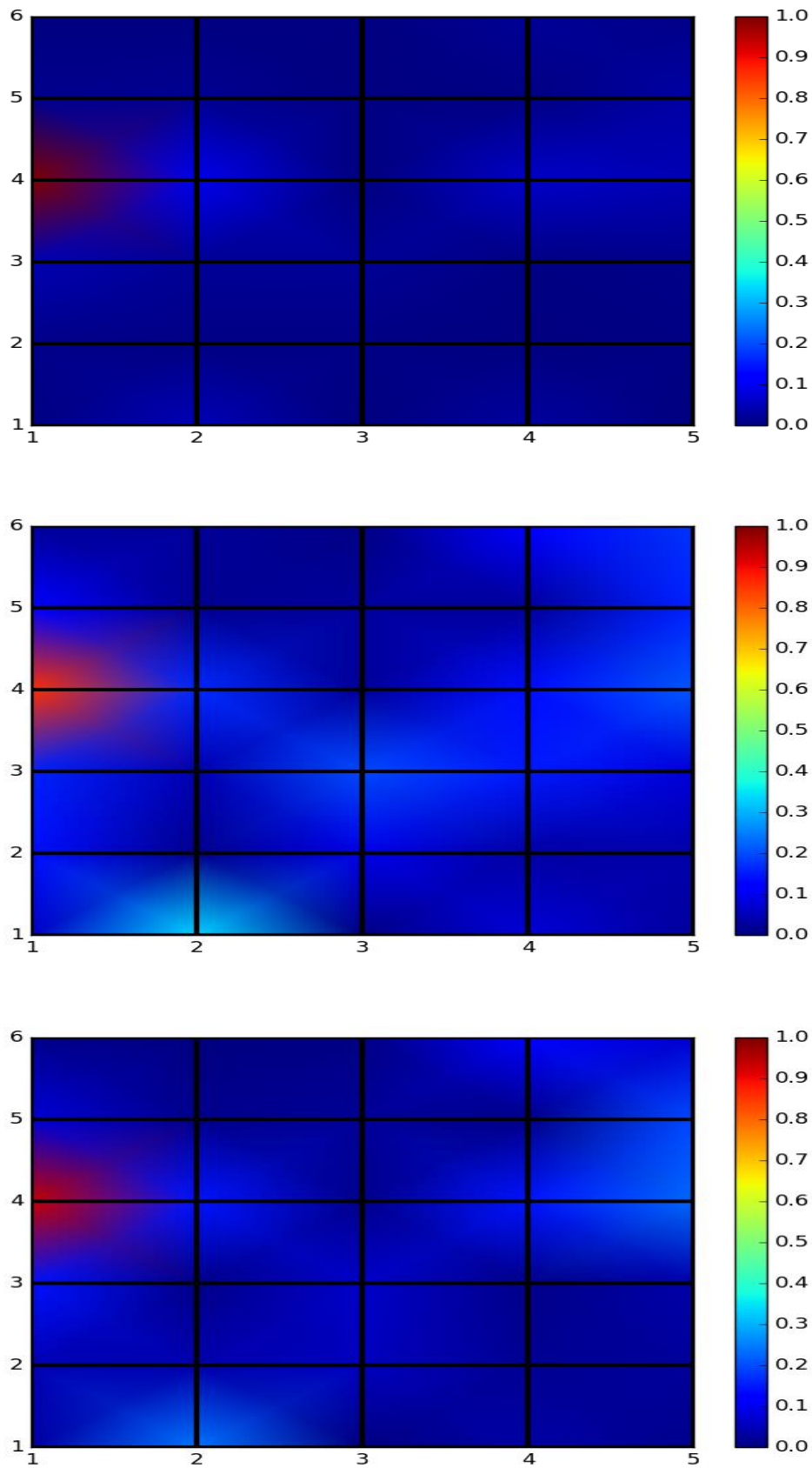


Fig. 15: Joint 16 Damaged

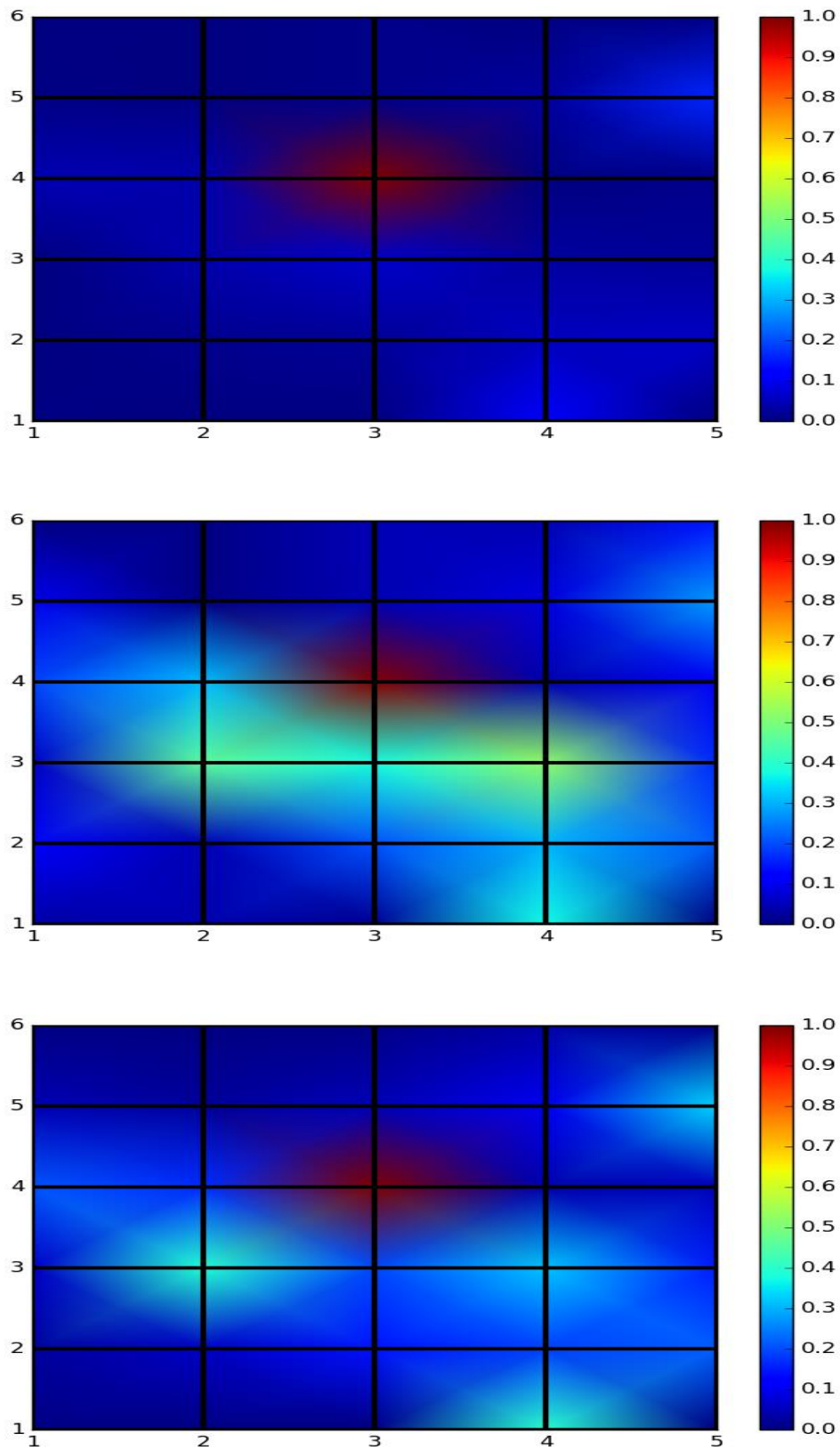


Fig. 16: Joint 18 Damaged

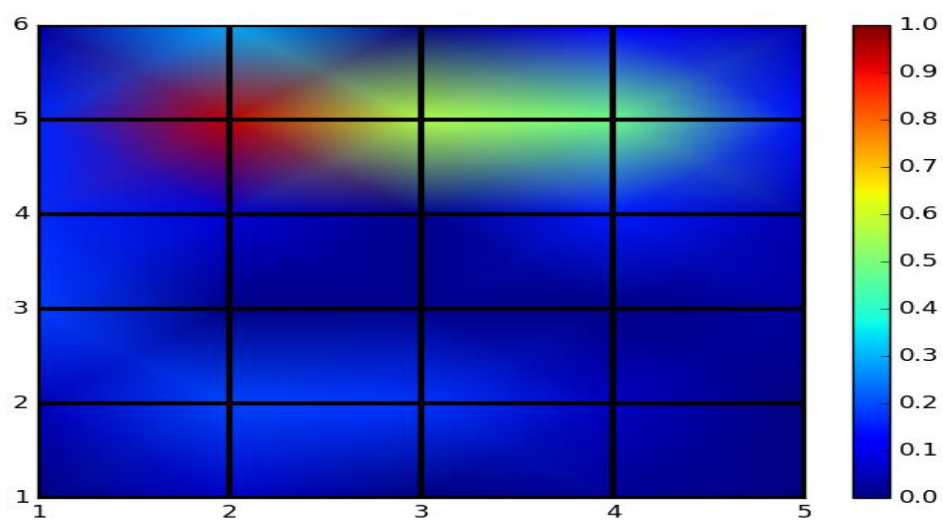
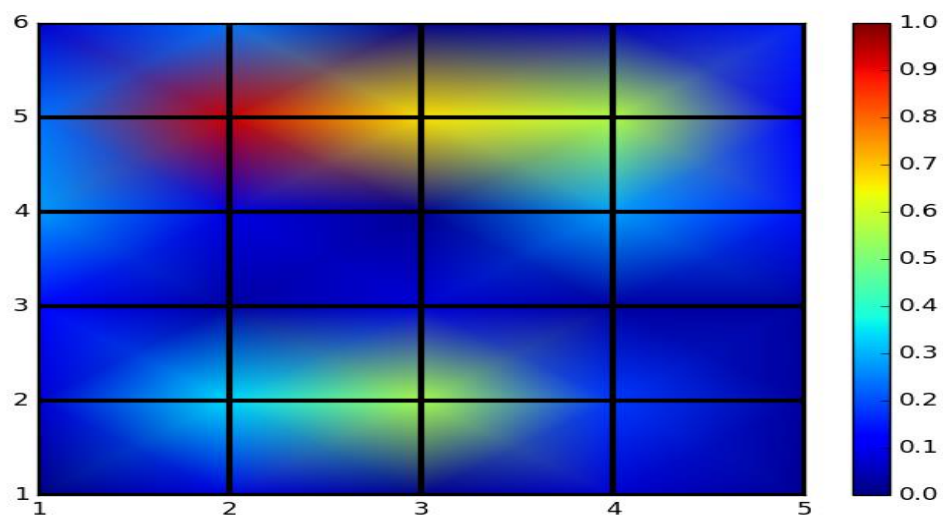
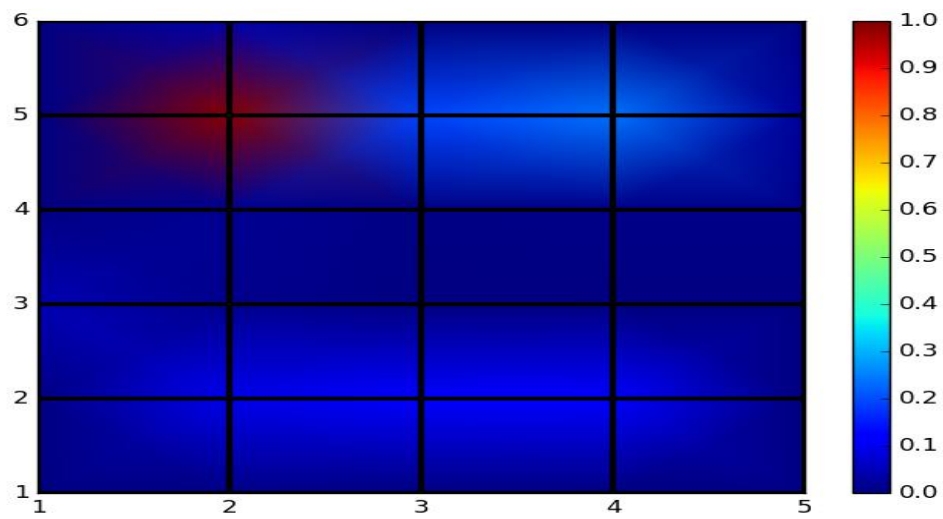


Fig. 17: Joint 22 Damaged

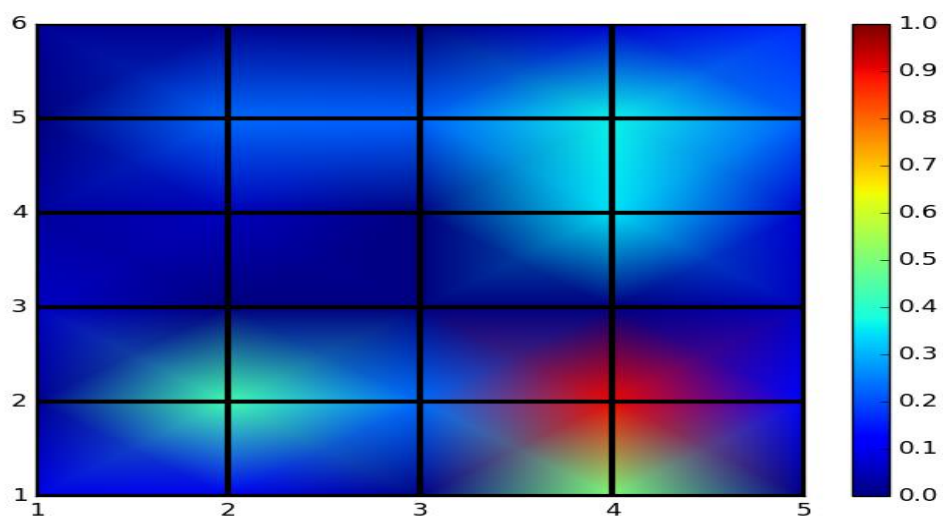
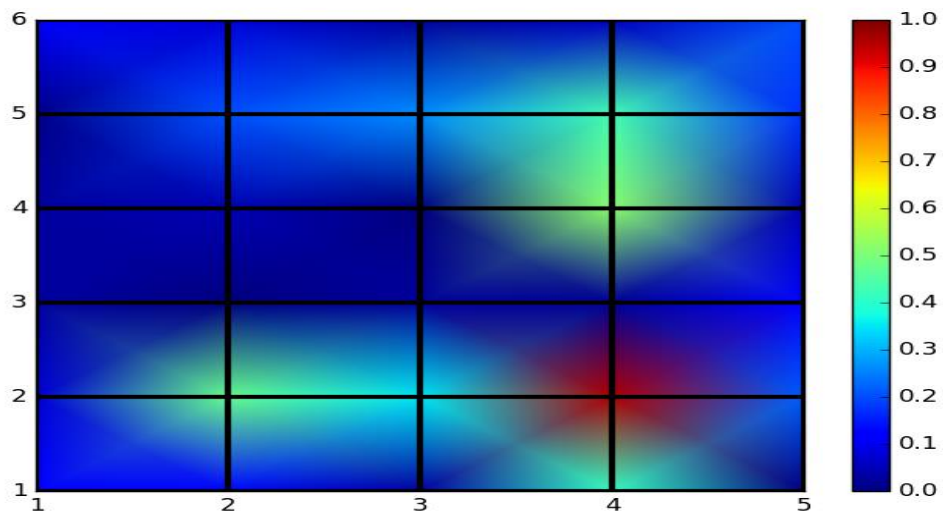
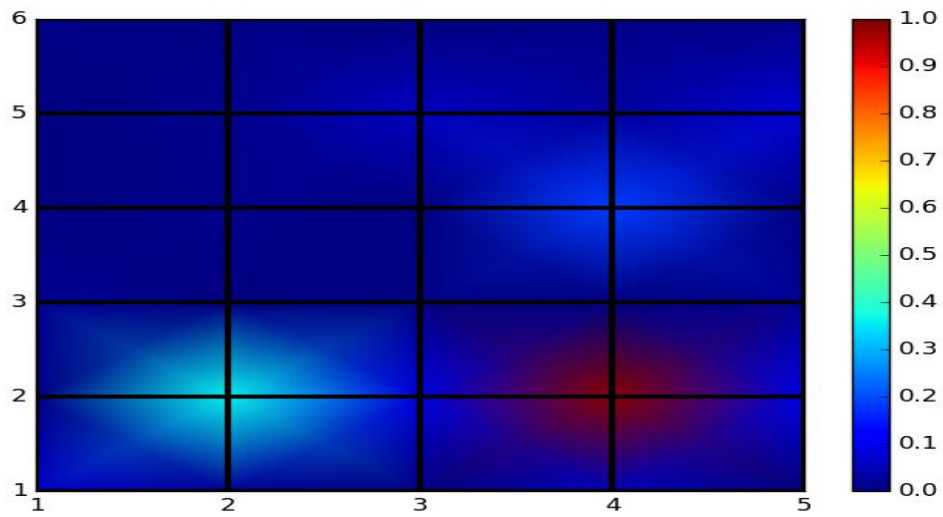


Fig. 18: Joint 9 Damaged

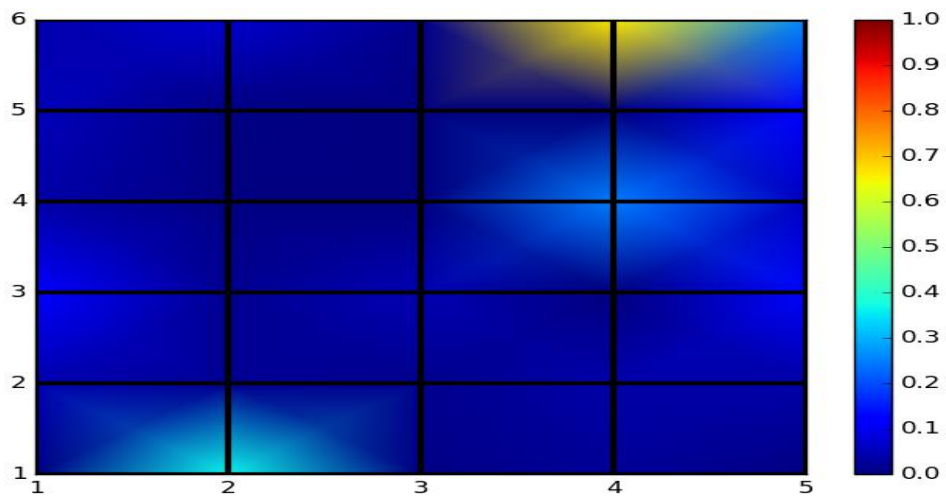
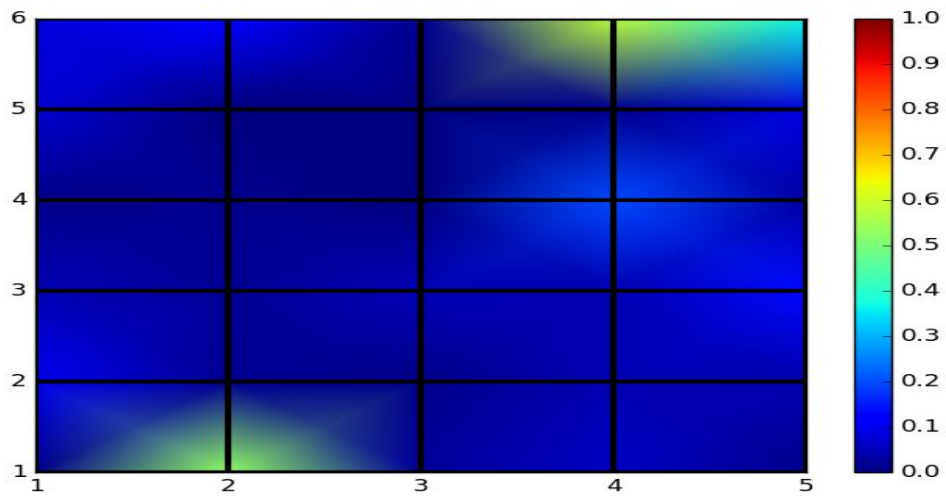
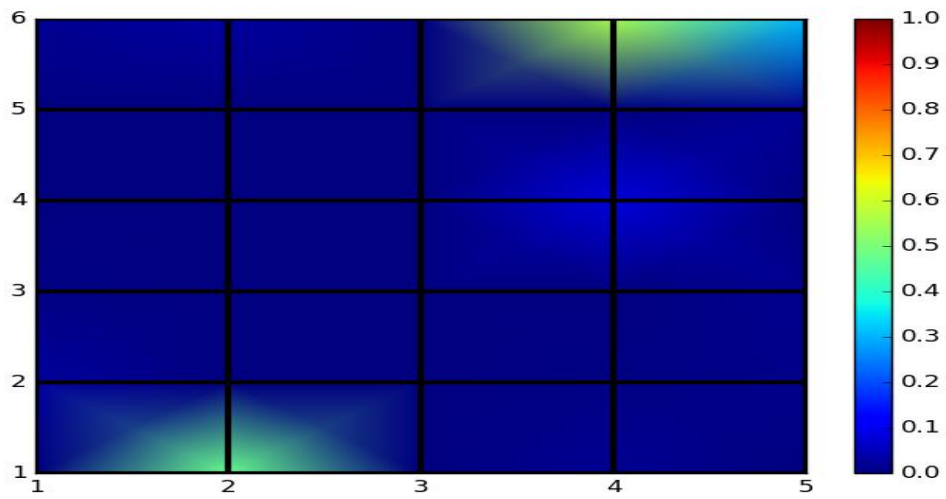


Fig. 19: Undamaged

References

1. Abdeljaber, O., Avci, O., Kiranyaz, S., Gabbouj, M., and Inman, D. J. (2017). ‘Real-time vibration-based structural damage detection using one-dimensional convolutional neural networks’. *Journal of Sound and Vibration*, 388, 154-170.
2. S Sony, S Gamage, A Sadhu, and J Samarabandu (2022). ‘Vibration-based multiclass damage detection and localization using long short-term memory networks’. *Structures* 35, 436-451.
3. Onur Avci. <http://onur-avci.com/>
4. Sumit Saha. *A Comprehensive Guide to Convolutional Neural Networks - the ELI5 way*. Medium <https://towardsdatascience.com/a-comprehensive-guide-to-convolutional-neural-networks-the-eli5-way-3bd2b1164a53>.
5. Christopher Olah. *Understanding LSTM Networks*. Github <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>
6. Christopher Olah. *Neural Networks, Types, and Functional Programming*. Github. <https://colah.github.io/posts/2015-09-NN-Types-FP/>
7. Samhita Alla. *Advanced Recurrent Neural Networks: Bidirectional RNNs*. Paperspace <https://blog.paperspace.com/bidirectional-rnn-keras/>
8. Etqad Khan. *LSTM: What’s the fuss about?* Medium. <https://medium.com/analytics-vidhya/lstm-whats-the-fuss-about-1ae9d4c3e33e>